Пропорциональое деление

В этой лекции разберем некоторые другие планировщики, известные под названием планировщики пропорционального деления или еще называемые планировщики равных долей (fair-share). Пропорциональное деление базируется на простой концепции: вместо того чтобы оптимизировать время оборота или время отклика, планировщик должен гарантировать, что каждая задача получит свой определенный процент процессорного времени.

Одна из ранних реализаций этой концепции известна как лотерейный планировщик. Основная идея такова: каждый раз проводите лотерею, чтобы определить какой процесс будет запущен следующим. Процессы, которые должны запускаться чаще должны иметь больше шансов на победу в лотерее.

Суть проблемы: как делить процессорное время пропорционально.
Как мы можем разработать планировщик так, чтобы он делил процессорное время пропорционально? Какие ключевые механизмы будут выполнять этот процесс? Насколько они будут эффективны?

Базовая концепция: билеты отражают вашу долю

В основе лотерейного планировщика лежит простая базовая концепция: билетики. Они используются для отражения доли ресурса, который процесс (или пользователь) должен получить. Сколько билетов у процесса на руках, такую долю ресурса он и получит.

В качестве примера представим два процесса A и B, и предположим, что у процесса A 75% всех билетов, а у B только 25%, и таким образом A должен получить 75% процессорного времени, а B только 25%.

Лотерейный планировщик достигает этих результатов вероятностно (но не определенно), проводя лотерею каждый некоторый период (например после каждого окончания временного окна). Проведение лотереи довольно просто: планировщик должен знать сколько всего есть билетиков (в нашем примере 100). Затем планировщик выбирает выигрышные номера 0-99. Предполагая, что у A билетики с номерами от 0-74, а у B 75-99, выигрышный билет просто определяет будет запущен процесс A или B. Планировщик затем просто загружает выигрышный процесс и запускает его.

Пример:

```
63 85 70 39 76 17 29 41 36 39 10 99 68 83 63 62 43 0 49 49
A       A A    A  A A  A A  A     A      A A A A A A
   B         B                    B       B
```

Из примера видно, что случайность лотереи приводит к некоторому предположительному ожиданию, но не гарантирует четкого исполнения пропорции. В этом примере B был запущен только 4 раза и получил 20% времени работы вместо 25%.

Однако, чем дольше будут работать задачи, тем ближе всего они будут к заданному проценту.

Билетный механизм

Лотерейный планировщик также предоставляет некоторые механизмы, позволяющие манипулировать билетами некоторыми другими, иногда полезными способами. **Один из таких путей** - концепция валюты. Валюта позволяет пользователю с набором билетом распределять их между своими задачами. Система затем автоматически конвертирует эту валюту в некоторое корректное глобальное значение.

Например предположим пользователи A и B имеют каждый по 100 билетов. ПОльзователь A запускает 2 задачи A1 и A2 и дает каждой из них по 500 билетов (суммарно получается 1000) в его собственной валюте. Пользователь Б запускает 1 задачу и дает ей 10 билетов (из 10). Система сконвертирует выданные задачам A1 и A2 билеты и из выданных 500 в валюте пользователя каждая задача получит по 50 в глобальной пропорции. Аналогично 10 билетов процесса B1 преобразуются в 100 глобальных билетов. Лотерея затем будет проведена между глобально распределенными билетами (200 в сумме), чтобы определить какой процесс будет запущен.

```
User A -> 500 (A's currency) to A1 ->  50 (global currency)
          -> 500 (A's currency) to A2 ->  50 (global currency)
User B ->  10 (B's currency) to B1 -> 100 (global currency)
```

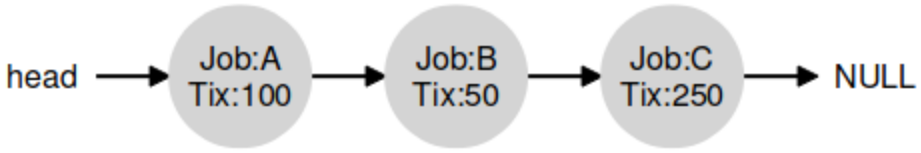
Другой полезный механизм - передача билетов. С возможностью передачи билетов, процесс может временно одолжить часть своих билетов другому процессу. Эта возможность особенно полезна в архитектуре клиент-сервер. Тогда, когда клиентский процесс ставит задачу серверному, то для увеличения скорости выполнения этой задачи клиентский процесс может отдать часть своих билетов серверному пока тот выполняет эту саму поставленную задачу для увеличения максимальной производительности. По окончанию задачи серверный процесс вернет билеты назад клиенту и все станет как прежде.

Наконец, билетная инфляция может иногда быть полезная. С инфляцией процесс может временно повышать или понижать количество персональных билетов. Естественно в среде, когда один процесс не доверяет другому такой номер не пройдет, потому что какой-нибудь недобросовестный процесс может захватить все время в системе. Но напротив подход может сработать когда процессы верят друг другу. В таком случае, если процесс знает о том что ему потребуется много процессорного времени может увеличить количество билетов на руках, при этом не взаимодействуя с каким бы то ни было другим процессом.

Реализация

Одна из наиболее восхитительных вещей в лотерейных планировщиках -простота реализации. Все что вам понадобится - хороший генератор случайных чисел для выбора выйгрышных номеров, структура данных для отслеживания состояния процессов в системе (например список) и знание об общем количестве билетов.

Представим что мы содержим структуру в списке. В этом примере будет 3 процесс A,B,C и у каждого свой набор билетов.



Для принятия решения о том какой процесс запускать, для начала нужно выбрать случайное число (победителя) из общего числа билетов (400). Например нам выпало число 300. Затем мы просто обходим список с некоторым счетчиком для того чтобы определить победителя. (код)

Этот код проходит через список процессов, добавляя значение каждого билетика к счетчику до тех пор пока полученное значение не превысит победивший билет. Когда это произойдет, значит текущий элемент списка и есть победитель. В нашем примере с билетом-победителем с номером 300 произойдет следующее. Во-первых счетчик увеличивается до 100 (задачи процесса A). 100<300, поэтому цикл продолжается. Затем счетчик становится равным 150 (задачи процесса B), и это все еще меньше 300. Прибавим 250 и получаем 400. 400>300 и таким образом мы выходим из цикла с переменной current указывающей на задачу C (победитель).

Для достижения большей эффективности этого процесса, лучше всего составлять списки процессов в отсортированном виде - от наибольшего числа билетов к наименьшему. Порядок сортировки не влияет на корректность алгоритма, и в целом сортировка помогает уменьшить количество операций для вычисления нужной задачи, особенно если несколько задач, которые владеют большинством билетов.

Пример

Для наилучшего понимания лотерейного планировщика разберем пример соперничества двух задач, каждая из которых будет иметь одинаковое количество билетов (100) и одинаковое время работы (R, время, которое мы и будем изменять).

В этом сценарии мы хотели бы чтобы каждая задача финишировала примерно в одно и тоже время, но из-за случайности лотереи предсказать как завершатся задачи несколько трудно. Для определения разницы времени завершения зададим метрику нечестности (unfairness), U. Метрикой будем считать время завершения первой задачи

поделенное на время завершения второй. Когда задачи завершаются примерно в одно и то же время, то метрика стремится к 1.

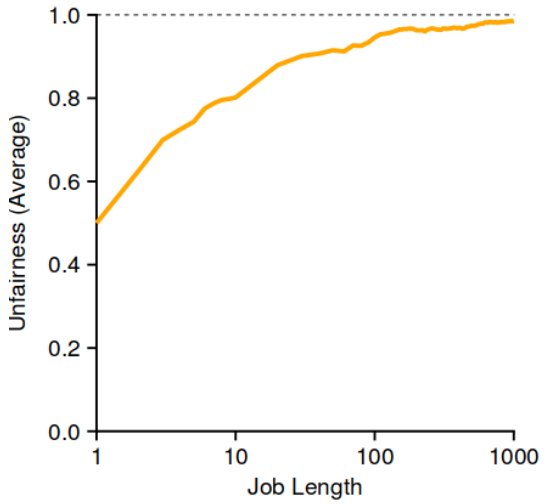


Figure 9.2: **Lottery Fairness Study**

График показывает среднюю нечестность в зависимости от времени работы двух задач. Как видно из графика, если время исполнения задачи невелико, средняя нечестность может быть довольно серьезной.

Как назначать билеты?

Одна из самых сложных тем в лотерейных планировщиках - как назначать билеты задачам? От этих назначений очень сильно будет зависеть производительность системы. Один из подходов к решению - предположение что пользователю лучше известно как распределять. В таком случае каждому пользователю выдается некоторое количество билетов и пользователь сам может распределять билеты задачам которые они запускают. Однако, это решение ни разу не решение, поскольку реально не дает порядок выполняемых действий. Таким образом проблема назначения билетов задачам так и остается открытой.

Почему не детерминистический?

Почему в качестве решения бы вовсе не рандомизировать все? Однако, как видно из примеров выше, случайность выбора дает нам простоту исполнения и приблизительную правильность, но к сожалению не достигает четкости пропорций, особенно на коротких временных промежутках. По этой причине Jean-Loup Waldspurger изобрел шагающий (stride) планировщик, который является детерминистическим, честно распределяющим планировщиком.

Шагающий планировщик тоже не сложен в своей концепции. Каждая задача в системе имеет шаг, значение которого противоположно числу билетиков, которые есть у этой задачи. В примере выше это было 100,50,250. Мы можем вычислить шаг, разделив

какое-то большое число на число билетиков назначенное процессу. Например, если мы разделим 10000 на число билетов, то получим для A: 100, для B:200 и для C:40. Полученное число называется **шагом** процесса. Каждый раз, когда процесс запускается мы будем увеличивать счетчик (называемый pass value) на значение шага процесса и таким образом отслеживать его глобальный прогресс.

Планировщик затем использует stride и pass для определения какой процесс запускать следующим. Основная идея проста: в любое время запускать тот процесс, который имеет наименьшее значение pass. После запуска процесса, увеличить pass value на значение шага.

```
current = remove_min(queue);        // pick client with minimum pass
schedule(current);                  // use resource for quantum
current->pass += current->stride;   // compute next pass using stride
insert(queue, current);             // put back into the queue
```

В нашем примере с 3 процессами с шагами 100,200,40 изначальное значение pass будет 0 для всех процессов. Первым будет запущен без разницы какой процесс, потому что значение pass у всех одинаково мало. Допустим процессорное время получит процесс A, после освобождения им процессора, pass value = 100. После запуска B, pass=200, после C, pass=40. В следующий момент алгоритм выберет снова процесс с наименьшим значением pass. Это будет процесс C. Отработав, Cpass=80. Что приведет к тому что C будет запущен снова. И тд.

Pass(A) (stride=100)	Pass(B) (stride=200)	Pass(C) (stride=40)	Who Runs?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	...

Figure 9.3: **Stride Scheduling: A Trace**

Результаты работы получаются следующие: процесс C отработает 5 раз, A дважды, и процесс B единожды, что согласно поставленным значениям 250,100,50 отражает точно картину пропорции. Таким образом лотерейное планирование достигает приблизительной пропорциональности с течением времени, а шаговый планировщик достигает четко поставленных значений.

Почему же тогда не использовать шаговый планировщик, если он достигает нужных значений? У шагового есть один серьезный недостаток по сравнению с лотерейным - он не имеет глобальной таблицы состояний процессов. В тот момент когда в середине работы некоторых процессов к планировщику поступит новый процесс, какое

значение pass у него должно быть? Если 0, то он монополизирует процессор. С лотерейным планировщиком при поступлении новой задачи мы просто добавляем ее в список, обновляем общее число билетов и продолжаем работу. В таком разрезе лотерейные планировщики гораздо более точны к распределению ресурсов для вновь поступивших процессов.

The Linux Completely Fair Scheduler (CFS)

Не смотря на эти ранние исследования в честно-распределяющих планировщиков, подход в текущих Linux-системах достигает подобных результатов, но несколько иначе. Планировщик называется Абсолютно честный Планировщик (CFS). Манера его работы похожа на честно-распределяющие планировщики, но достигается это в гораздо более эффективном и масштабируемом виде.

Для достижения целей эффективности, CFS проводит очень мало времени за принятием решений благодаря своему дизайну и разумному использованию структур данных, хорошо подходящих к процессу. Последние исследования показали что оказывается эффективность планировщика важна. В частности в ДЦ Гугла было доказано, что распределение задач пользователей занимает около 5% всего процессорного времени всего ДЦ. Уменьшение этих накладных расходов как можно больше - ключевая цель в современных архитектурах планировщиков.

Базовые операции

Тогда как большинство планировщиков базируются на концепции фиксированного временного окна, CFS работает несколько иначе. Его цель проста: честно и равномерно разделять CPU между всеми процессами. Достигается это с помощью техники основанной на подсчете (counting-based), называемой виртуальное время выполнения [virtual runtime (vruntime)].

В процессе своей работы программа накапливает vruntime. В самом общем виде vruntime увеличивается в том же темпе, пропорционально физическому (реальному) времени, проведенному на процессоре. Когда требуется решить какой процесс запускать следующим, CFS выберет процесс с наименьшим значением vruntime.

Возникает закономерный вопрос, когда планировщик узнает что нужно прервать текущий процесс и передать управление следующему? Если переключаться слишком часто, честность увеличивается, но происходит это ценой потери производительности (переключение контекстов). Если же переключаться редко, то повысится производительность, но ценой меньшей честности.

CFS решает эти вопросы через различные параметры. Один из таких параметров - **sched_latency**. CFS использует этот параметр для определения как долго процесс должен работать перед тем как рассмотреть необходимость переключения (эффективное определение окна времени, но в динамической манере). Типичное значение этого параметра - 48ms. CFS делит это число на количество запущенных процессов в системе

процессов чтобы определить размер окна времени для процесса и таким образом за период в указанном параметре алгоритм раздавал процессорное время честно.

Например, если в системе 4 процесса, CFS поделит значение на число процессов и получит что каждый из них должен работать по 12мс. Отработав первые 12 с процессом A планировщик выберет процесс с меньшим значением vruntime и даст процессорное время ему. После того как все процессы отработали по 2 такта, два процесса завершают работу, а оставшиеся делят время между собой, получая по 24мс времени.

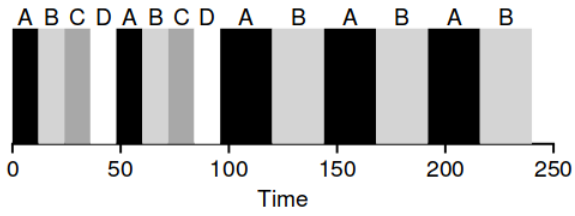


Figure 9.4: CFS Simple Example

Однако, что будет если процессов в системе много? Не приведет ли это к слишком маленькому окну и таким образом к большой нагрузке за счет контекстного переключения? Да, так и есть. Чтобы решить эту проблему, в CFS есть параметр min_granularity, который обычно равен 6мс. CFS никогда не выставит временное окно меньше чем это значение. Таким образом избегая оверхеда.

К примеру, если в системе будет 10 процессов, то временной интервал должен быть 4,8мс но CFS выставит в 6мс. И хотя честность за заданный интервал из-за этого пострадает, она пострадает не сильно, позволяя все еще достигать эффективность использования CPU.

Отметим, что CFS используют таймер прерываний, что означает, что решения CFS может принимать только через эти фиксированные интервалы. Это прерывание случается часто, например каждую 1мс, давая шанс CFS проснуться и определить не достигла ли текущая задача конца своей работы. Если у какой-либо задачи временное окно не делится нацело на время прерывания по таймеру, то это вполне допустимо - CFS отслеживает точное время vruntime, что означает что на длительном промежутке работы будет достигнуто идеальное распределение процессорного времени.

Weighting (Niceness)

CFS также позволяет иметь контроль над приоритетами, позволяя пользователям или администраторам предоставлять программам больше процессорного времени. Достигается это не с помощью билетов, а с помощью классического UNIX механизма, известного как nice уровень процесса. Параметр может быть выставлен в интервале от -20 до +19 со значением по-умолчанию, равному 0. Положительные значения nice **уменьшают** приоритет, а отрицательные - увеличивают.

CFS проектирует значения nice каждого процесса как вес:

```
static const int prio_to_weight[40] = {
 /* -20 */     88761,     71755,     56483,     46273,     36291,
 /* -15 */     29154,     23254,     18705,     14949,     11916,
 /* -10 */      9548,      7620,      6100,      4904,      3906,
 /*  -5 */      3121,      2501,      1991,      1586,      1277,
 /*   0 */      1024,       820,       655,       526,       423,
 /*   5 */       335,       272,       215,       172,       137,
 /*  10 */       110,        87,        70,        56,        45,
 /*  15 */        36,        29,        23,        18,        15,
};
```

Эти веса позволяют нам вычислять время окна каждого процесса, но теперь принимая во внимание различие их приоритетов.
Используется вот такая формула для вычисления:

$$\texttt{time_slice}_k = \frac{\texttt{weight}_k}{\sum_{n=0}^{n-1} \texttt{weight}_i} \cdot \texttt{sched_latency} \qquad (9.1)$$

$$\texttt{vruntime}_i = \texttt{vruntime}_i + \frac{\texttt{weight}_0}{\texttt{weight}_i} \cdot \texttt{runtime}_i \qquad (9.2)$$